

1. Descripción del proyecto y ámbito de implantación

UltraPresetGaming es una aplicación web orientada a la comparación de precios de hardware y videojuegos entre distintas tiendas especializadas y plataformas de venta online. Su objetivo principal es permitir al usuario identificar, de manera rápida y visual, en qué tienda puede adquirir un determinado producto al mejor precio, manteniendo siempre una referencia frente a un precio oficial o recomendado. El proyecto se enmarca en el contexto del Trabajo Intermodular de 2.º de Desarrollo de Aplicaciones Web, siguiendo la modalidad de **Tipo 2: HTML, CSS y JavaScript consumiendo un API REST de elaboración propia**.

La aplicación está dirigida principalmente a usuarios entusiastas del gaming y del hardware de PC que desean optimizar su presupuesto a la hora de montar o actualizar un equipo, así como a compradores que buscan comparar rápidamente el coste de un videojuego en diferentes tiendas sin tener que revisar manualmente cada página. También puede resultar útil para usuarios menos expertos que simplemente quieren una visión clara y unificada de precios, sin tener que interpretar listados complejos o interfaces distintas en cada comercio.

Desde el punto de vista tecnológico, UltraPresetGaming se compone de un frontend desarrollado con HTML5, CSS3 y JavaScript vanilla, que consume una API REST diseñada con Django y Django REST Framework. El backend expone recursos como productos, tiendas, ofertas y alertas de precio a través de diferentes endpoints HTTP (GET, POST, PUT, PATCH y DELETE), utilizando JSON como formato de intercambio de datos, tal y como se exige en los requisitos del proyecto Tipo 2. En la parte de persistencia se utiliza una base de datos relacional, donde se modelan, entre otras, las entidades Producto, Tienda, Oferta y AlertaPrecio, incluyendo una relación ternaria que vincula usuario, producto y tienda para la gestión de alertas de bajada de precio.

Uno de los elementos diferenciales del proyecto es la integración de un motor de scraping avanzado, implementado con Playwright, que se encarga de visitar periódicamente distintas tiendas de hardware y videojuegos (como PcComponentes, Coolmod, Life Informática, Alternate o NeoByte), extraer sus listados de productos y ofertas, y almacenar los datos de forma estructurada en la base de datos. Sobre esta capa de scraping se ha construido un sistema de emparejamiento inteligente de productos que combina técnicas de similitud textual mediante TF-IDF con el uso de modelos de inteligencia artificial locales ejecutados a través de Ollama. Este sistema permite decidir si una oferta recién encontrada corresponde realmente al mismo producto que ya existe en el catálogo, reduciendo duplicidades y mejorando la calidad de la información mostrada en el comparador.

En cuanto a la evolución futura, el diseño del proyecto se ha planteado de forma modular para facilitar su ampliación. Resulta relativamente sencillo incorporar nuevas tiendas o plataformas, así como nuevas categorías de producto, simplemente añadiendo nuevos scrapers especializados y ajustando la lógica de normalización de datos. Del mismo modo, la arquitectura de la API REST permite añadir nuevos endpoints o funcionalidades (como recomendaciones personalizadas, históricos de precio o sistemas de notificaciones) sin

romper la estructura existente. Por todo ello, el proyecto no solo cumple los requisitos académicos establecidos, sino que sienta las bases para una posible evolución hacia un producto real de comparativa de precios orientado al sector gaming.

2. Temporalización del proyecto y fases del desarrollo

El proyecto UltraPresetGaming se ha desarrollado a lo largo del curso académico, siguiendo una planificación por fases que combina tareas de análisis, diseño, implementación, pruebas y documentación. La duración total aproximada se ajusta a la estimación de 50 horas de trabajo indicada para el proyecto intermodular, distribuidas de forma equilibrada entre frontend, backend, scraping e integración de inteligencia artificial.

A continuación, se presenta un resumen de las fases principales y su temporalización relativa. Las semanas indicadas son orientativas y se refieren al calendario del módulo de Proyecto Intermodular, desde el inicio de la planificación hasta la entrega final.

2.1 Planificación y fases del desarrollo

Fase	Descripción	Duración aproximada
1. Análisis y diseño inicial	Definición de la idea de comparador de precios, elección del Tipo 2 (HTML+CSS+JS + API REST Django), revisión de requisitos y alcance.	Semanas 1–2
2. Diseño de interfaz y estructura de pantallas	Elaboración de wireframes, definición de páginas principales (inicio, hardware, videojuegos, perfil, login, precios), estructura de navegación y criterios de responsive.	Semanas 2–3
3. Diseño de base de datos y modelos Django	Modelado de entidades (Producto, Tienda, Oferta, Perfil, ItemGuardado, AlertaPrecio, DecisionIA), definición de relaciones 1:N, N:M y ternaria, creación de migraciones iniciales.	Semanas 3–4
4. Implementación inicial de la API REST	Creación de la app api, serializers, viewsets y rutas básicas; exposición de endpoints CRUD para productos, tiendas y ofertas, uso de JSON y parámetros en ruta, query y cuerpo.	Semanas 4–5

5. Desarrollo del frontend consumiendo la API	Maquetación detallada de páginas HTML, aplicación de estilos CSS modulares, integración de JavaScript para consumir la API (listados de productos, login/registro, perfil, carrito).	Semanas 5–7
6. Implementación del scraper básico	Creación del proyecto scrapper_app, definición de BaseScraper, scrapers por tienda (PcComponentes, Coolmod, LifeInformática, Alternate, NeoByte), orquestador main_crawler.py y primer guardado de productos/ofertas.	Semanas 6–8
7. Optimización de scraping e IA	Integración de TF-IDF en db_manager.py, optimización de llamadas a IA (batching, fallback de modelos, caché en DecisionIA), logging centralizado, graceful shutdown y mejoras de rendimiento en BD (bulk_create, índices, soft delete).	Semanas 8–10
8. Pruebas, corrección de errores y mejoras de UX	Pruebas de extremo a extremo (scraper + API + frontend), ajuste de rutas de imágenes, coherencia en la navegación, refinado de estilos y textos, corrección de errores detectados durante el uso real.	Semanas 10–11
9. Documentación y memoria	Redacción del README, elaboración de diagramas (Gantt, Pert, SQL, pantallas), escritura de la memoria en PDF y preparación de material para la defensa.	Semanas 11–12

Esta planificación refleja tanto la secuencia lógica del desarrollo (primero definir el modelo de datos y la API, luego el frontend y finalmente el scraping avanzado) como la necesidad de iterar sobre las funcionalidades una vez disponibles los primeros datos reales de las tiendas.

2.2 Diagrama de Gantt (descripción)

El diagrama de Gantt asociado a este proyecto representa en el eje horizontal las semanas de trabajo y, en el eje vertical, las fases descritas en la tabla anterior. Cada fase se muestra como una barra cuyo inicio y fin coinciden con las semanas de planificación estimadas, permitiendo visualizar tanto la duración de cada actividad como los solapamientos entre ellas.

Por ejemplo, mientras se finalizaba el diseño de la base de datos (fase 3), se pudo avanzar en paralelo en los primeros bocetos de la interfaz (fase 2). De manera similar, el desarrollo inicial del frontend (fase 5) se solapó con las primeras pruebas del scraper básico (fase 6),

permitiendo comprobar cuanto antes el flujo completo desde la captura de datos hasta su visualización en la web. Este solapamiento de tareas es especialmente visible en la franja de semanas 5 a 8, donde backend, frontend y scraping evolucionan de forma coordinada.

2.3 Diagrama de Pert (descripción)

El diagrama de Pert complementa al Gantt mostrando las dependencias entre tareas mediante nodos y flechas. En el contexto de UltraPresetGaming, los hitos más relevantes del Pert pueden agruparse de la siguiente manera:

- Nodo inicial: definición de la idea y elección del tipo de proyecto (Fase 1).
- Nodo de diseño de datos: finalización del modelo de base de datos y migraciones principales (Fase 3), que depende del análisis inicial.
- Nodo de API mínima viable: primeros endpoints funcionales para productos y tiendas (Fase 4), dependiente del diseño de modelos.
- Nodo de frontend conectado: páginas de hardware y videojuegos consumiendo la API (Fase 5), dependiente de la API mínima viable.
- Nodo de scraper básico: obtención de los primeros datos reales desde las tiendas (Fase 6), dependiente del diseño de datos y de la estructura general del backend.
- Nodo de optimización de IA y rendimiento: integración de TF-IDF, caché de decisiones, bulk operations y graceful shutdown (Fase 7), dependiente del scraper básico y de la API ya estable.
- Nodo de pruebas y estabilización: validación del flujo completo (scraper → BD → API → frontend) (Fase 8).
- Nodo final: documentación y preparación de la memoria y la defensa (Fase 9).

Las flechas del Pert reflejan que no es posible, por ejemplo, optimizar las decisiones de IA sin disponer previamente de un conjunto suficiente de datos reales en la base de datos, o que el frontend no puede integrarse con el comparador de precios hasta que la API proporcione ofertas estructuradas y filtradas correctamente.

2.4 Retos encontrados y aprendizajes significativos

A lo largo del desarrollo del proyecto se han encontrado diversos retos técnicos que han generado aprendizajes importantes. Uno de los primeros desafíos estuvo relacionado con el rendimiento en el guardado de datos durante el scraping: al trabajar con un volumen elevado de productos y ofertas, el uso de operaciones de guardado individuales generaba bloqueos y tiempos de respuesta elevados en la base de datos SQLite. La solución vino de la mano de operaciones masivas (`bulk_create` y `bulk_update`), combinadas con un diseño cuidadoso de los índices y de la estrategia de soft delete mediante el campo disponible, reduciendo drásticamente el tiempo de bloqueo y mejorando la escalabilidad del sistema.

Otro reto clave fue la integración de la inteligencia artificial para el emparejamiento de productos. Inicialmente, las llamadas directas a modelos locales de lenguaje resultaban

costosas en tiempo y recursos, especialmente cuando se realizaban de forma aislada por cada candidato. Se introdujo entonces un motor híbrido: por un lado, un sistema de similitud textual basado en TF-IDF y similitud del coseno para filtrar de forma rápida candidatos obvios; por otro, un flujo de IA con batching de peticiones, modelo ligero como filtro previo y modelo más pesado solo para casos dudosos, junto con una tabla de caché (DecisionIA) que almacena resultados ya evaluados. Este enfoque ha permitido reducir de forma significativa la carga de cómputo y el tiempo total de ejecución del scraper.

También resultó especialmente desafiante garantizar la estabilidad del scraping frente a las medidas de protección de las tiendas (WAF, captchas, cambios en el DOM, etc.). Para afrontarlo, se implementaron técnicas avanzadas como la rotación dinámica de User-Agents y Client-Hints, la simulación de comportamiento humano (movimientos de ratón, scroll, tiempos de espera irregulares) y la detección y resolución de pantallas intermedias tipo “Just a moment...”. Además, se añadió un sistema de reintentos en determinadas interacciones (por ejemplo, al pulsar botones de “Cargar más”), acompañado de un graceful shutdown que asegura el cierre ordenado de hilos, contextos de Playwright y conexiones a base de datos en caso de interrupción.

Por último, la integración completa entre scraper, backend y frontend supuso un aprendizaje significativo en cuanto a diseño modular y mantenimiento. Fue necesario reorganizar carpetas, unificar estilos, ajustar rutas de imágenes, garantizar que la API devolviese únicamente ofertas activas y que el frontend pudiera manejar tanto carritos de usuario autenticado como carritos de invitado usando localStorage. Este trabajo de pulido final ha permitido que la experiencia de usuario sea más coherente y que el código resultante sea más legible y mantenible de cara a futuras ampliaciones.

3. Requisitos hardware y software

Para garantizar el correcto funcionamiento de UltraPresetGaming es necesario definir unos requisitos mínimos y recomendados tanto de hardware como de software. Estos requisitos dependen, principalmente, de si se desea ejecutar únicamente la aplicación web (frontend + API REST) o si, además, se va a utilizar el motor de scraping y los modelos de inteligencia artificial locales integrados mediante Ollama.

3.1 Requisitos hardware

3.1.1 Entorno de desarrollo y pruebas (completo: API + frontend + scraping + IA local)

En el contexto del desarrollo del proyecto se ha trabajado con un entorno local que ejecuta simultáneamente el servidor Django, la base de datos, el motor de scraping con Playwright y

modelos de IA locales a través de Ollama. Este escenario es el más exigente en términos de recursos, por lo que se establecen los siguientes requisitos recomendados:

- **Procesador (CPU)**
 - Mínimo: CPU de 4 núcleos (por ejemplo, Intel Core i5 o AMD Ryzen 5) para poder ejecutar de forma razonable el servidor web, el scraper multihilo y el navegador automatizado.
 - Recomendado: CPU de 6–8 núcleos (Intel Core i7 o AMD Ryzen 7) para mejorar la capacidad de concurrencia durante el scraping y el uso de modelos de IA locales.
- **Memoria RAM**
 - Mínimo: 16 GB de RAM para trabajar con modelos pequeños/medios en Ollama y a la vez ejecutar el servidor Django, el entorno de desarrollo y Playwright sin bloqueos frecuentes. Diversas guías de uso de modelos locales señalan 16 GB como un punto de partida razonable para este tipo de cargas.
 - Recomendado: 24–32 GB de RAM para poder manejar modelos de mayor tamaño, abrir varios navegadores de Playwright en paralelo y disponer de margen para herramientas adicionales (IDE, navegador, etc.).
- **Almacenamiento**
 - Mínimo: 50 GB de espacio libre en disco SSD para alojar el entorno virtual de Python, dependencias, base de datos SQLite, recursos estáticos del frontend y uno o varios modelos ligeros de Ollama.
 - Recomendado: 100 GB o más si se planea descargar modelos adicionales o conservar históricos de datos de scraping a largo plazo.
- **Tarjeta gráfica (GPU)**
 - Opcional: UltraPresetGaming no requiere GPU dedicada para funcionar, pero disponer de una GPU compatible con CUDA o con los controladores soportados por Ollama permite acelerar la ejecución de modelos de lenguaje y mejorar los tiempos de respuesta del sistema de IA.

3.1.2 Entorno de ejecución ligero (solo API + frontend, sin IA local)

En un entorno donde únicamente se ejecute la API REST de Django y el frontend (por ejemplo, en un servidor de despliegue o en una máquina que no necesite lanzar el scraper ni los modelos locales), los requisitos de hardware son significativamente menores:

- **Procesador (CPU):** CPU de 2–4 núcleos es suficiente para servir la API y responder a peticiones de usuarios con carga moderada.
- **Memoria RAM:** entre 2 GB (mínimo) y 4 GB (recomendado) para ejecutar Django, la base de datos y el servidor web (por ejemplo, Gunicorn + Nginx) sin saturaciones en un escenario de uso académico.
- **Almacenamiento:** alrededor de 10–20 GB libres para el código, las dependencias y los datos de la base de datos, especialmente si se migra de SQLite a un motor más robusto en entornos productivos.

En ambos casos se recomienda utilizar unidades SSD en lugar de discos mecánicos, a fin de mejorar los tiempos de arranque y el acceso a los ficheros de proyecto y modelos.

3.2 Requisitos software

Los requisitos software se dividen en dos bloques: por un lado, las herramientas necesarias para el desarrollo y ejecución del backend en Django y la API REST; por otro, las dependencias relacionadas con el scraping y la inteligencia artificial local.

3.2.1 Backend y API REST (Django + Django REST Framework)

Para el backend se han utilizado las tecnologías habituales en el ecosistema Django. La propia documentación oficial indica que Django únicamente requiere disponer de una versión de Python compatible y, para desarrollo, puede ejecutarse con el servidor integrado y una base de datos como SQLite.

- **Lenguaje y runtime**
 - Python 3.10 o superior, instalado en el sistema.
 - Gestor de paquetes pip para instalar las dependencias del proyecto.
- **Frameworks y librerías principales**
 - Django (versión 4.x o superior), como framework principal del backend.
 - Django REST Framework, para la construcción de la API REST y la serialización de datos.
 - SQLite como base de datos por defecto en desarrollo; el proyecto es adaptable a otros motores como PostgreSQL en caso de despliegue en producción.
- **Entorno de desarrollo**
 - Sistema operativo compatible con Python 3.10+ (Windows 10/11, macOS, o distribuciones Linux modernas).
 - Entorno virtual (venv) recomendado para aislar las dependencias del proyecto.
 - Un editor o IDE orientado a desarrollo web (Visual Studio Code, PyCharm u otros).

3.2.2 Scraping con Playwright

Para el motor de scraping se ha utilizado Playwright para Python, que requiere cumplir unos requisitos mínimos de versión de Python y sistema operativo, especialmente en la parte de instalación de los navegadores automatizados.

- **Requisitos de Playwright (Python)**
 - Python 3.8 o superior.
 - Sistemas soportados:

- Windows 11 o Windows Server 2019 (o superior), incluyendo Windows Subsystem for Linux (WSL).
- macOS 14 Ventura o versiones posteriores.
- Distribuciones Linux modernas, como Debian 12/13 o Ubuntu 22.04/24.04, sobre arquitecturas x86-64 o ARM64, con algunas dependencias adicionales para ejecutar los navegadores incluidos.
- **Dependencias asociadas**
 - Instalación de los binarios de navegador (Chromium, Firefox, WebKit) mediante el comando `python -m playwright install`.
 - Posibles librerías del sistema adicionales en Linux (como fuentes, bibliotecas de gráficos y multimedia) según recomiendan las imágenes base oficiales de Playwright.

3.2.3 Inteligencia artificial local con Ollama

UltraPresetGaming integra un módulo de IA que aprovecha modelos de lenguaje ejecutados en local mediante Ollama para tomar decisiones sobre el emparejamiento de productos y la validación de resultados del motor de similitud. Ollama proporciona binarios para Windows, macOS y Linux, y mantiene una biblioteca de modelos con distintos tamaños de parámetros y ficheros de varios gigabytes.

- **Instalación básica de Ollama**
 - macOS: instalación mediante paquete oficial descargado desde la web de Ollama.
 - Windows: instalador oficial para Windows 10/11.
 - Linux: script de instalación (`curl -fsSL https://ollama.com/install.sh | sh`) o imágenes Docker oficiales.
- **Requisitos generales**
 - 8 GB de RAM como mínimo y 16 GB recomendados para trabajar con modelos de tamaño medio (7B–13B parámetros) de forma fluida.
 - Espacio libre en disco suficiente para almacenar los modelos descargados; por ejemplo, modelos de entre 3 y 9 GB para variantes tipo Phi, Mistral o Llama de tamaño intermedio, tal y como se indica en la propia biblioteca de modelos de Ollama.
 - Opcionalmente, una GPU compatible (NVIDIA o AMD con drivers adecuados) para acelerar la inferencia, aunque el proyecto está preparado para funcionar también en modo CPU-only, priorizando modelos ligeros cuando el hardware es más limitado.

3.2.4 Frontend (HTML, CSS y JavaScript)

La parte de frontend no introduce requisitos especiales más allá de un navegador moderno y compatible con HTML5, CSS3 y JavaScript. Esto incluye los navegadores habituales

(Google Chrome, Microsoft Edge, Mozilla Firefox, Safari o equivalentes) en sus versiones actuales.

La aplicación está desarrollada con JavaScript vanilla, por lo que no requiere la instalación de frameworks adicionales en el cliente. Para servir los ficheros estáticos durante el desarrollo es suficiente con utilizar el propio servidor de Django o una extensión como Live Server en Visual Studio Code.

4. Arquitectura de la aplicación

La arquitectura de UltraPresetGaming se ha diseñado siguiendo una separación clara entre frontend y backend, con una API REST como capa de comunicación entre ambos. En la parte cliente se emplean HTML5, CSS3 y JavaScript vanilla para construir la interfaz y gestionar la interacción con el usuario, mientras que en la parte servidor se utiliza Django junto con Django REST Framework para exponer los recursos de la base de datos y orquestar la lógica de negocio. Adicionalmente, el proyecto incluye un módulo de scraping independiente que se integra con el backend mediante el acceso directo a los modelos y a la base de datos.

4.1 Arquitectura general y organización de carpetas

La raíz del proyecto se organiza en tres bloques principales: backend, frontend y scraper_app, además de carpetas auxiliares como docs y scripts. El directorio backend contiene el proyecto Django (comparador), la aplicación principal de API (api), la base de datos SQLite y los ficheros de configuración; el directorio frontend agrupa las páginas HTML, las hojas de estilo y los scripts JavaScript; y el directorio scraper_app aloja el motor de scraping, los scrapers específicos por tienda y los utilitarios de IA y base de datos.

Esta estructura modular permite mantener separadas las responsabilidades de cada capa: la interfaz se desarrolla y prueba de forma independiente, la API se encarga de exponer datos de forma coherente y segura, y el scraper puede evolucionar sin necesidad de modificar el código del frontend. La carpeta docs se reserva para documentación y anexos (incluyendo los PDFs con los requisitos y la explicación del proyecto intermodular), mientras que scripts contiene utilidades PowerShell para facilitar la ejecución del crawler y tareas de apoyo al desarrollo.

4.2 Frontend: HTML, CSS y JavaScript

El frontend se encuentra bajo el directorio frontend y sigue una organización por páginas, apoyada en recursos compartidos. Dentro de frontend/pages se agrupan las distintas pantallas de la aplicación: por ejemplo, home (página principal), hardware, videogames, prices, profile, login, register, news y mods, cada una con su correspondiente par de ficheros .html, .css y .js

cuando procede. Esta organización facilita que cada pantalla tenga su propia hoja de estilos y su script de comportamiento, manteniendo una separación clara de responsabilidades.

Las páginas emplean etiquetas semánticas de HTML5 como header, nav, main, section, article y footer para estructurar el contenido, mejorar la accesibilidad y facilitar la comprensión del marcado. La carpeta frontend/shared/css contiene estilos comunes, tales como reset.css, header.css, footer.css, button.css y profile.css, que se reutilizan en múltiples páginas. De este modo, se evitan duplicidades y se centralizan elementos como el diseño del encabezado, el pie de página, los botones o la maquetación del área de perfil.

La lógica de autenticación y elementos compartidos del encabezado se encapsula en frontend/shared/js/auth-header.js. Este script se encarga de leer el estado de sesión (por ejemplo, el token JWT o la información de usuario), modificar dinámicamente el menú de navegación y adaptar la interfaz según si el usuario está autenticado o no. En las páginas de negocio, como hardware.js, videogames.js o prices.js, se realizan peticiones fetch a la API REST para cargar listados de productos, ofertas y configuraciones de carrito, aplicando filtros y actualizando el DOM de forma dinámica.

En cuanto al diseño responsive, el proyecto hace uso de media queries en las distintas hojas de estilo para asegurar que el sitio se adapta adecuadamente a dispositivos móviles, tablets y pantallas de escritorio. Se ajustan tamaños de fuentes, rejillas de productos, disposición de tarjetas, barras de navegación y elementos de carrito para mantener la usabilidad en diferentes resoluciones, tal y como se exige en los requisitos del proyecto Tipo 2.

4.3 Backend: Django, modelos y diagrama SQL

El backend se organiza en torno al proyecto Django comparador y la aplicación api, situados dentro del directorio backend. El proyecto comparador incluye la configuración principal (settings.py), la definición de rutas de alto nivel (urls.py) y los puntos de entrada ASGI/WSGI, mientras que la app api concentra los modelos, serializers, vistas y rutas específicas de la API REST.

Dentro de backend/api/models.py se definen las entidades principales del dominio: Producto, Tienda, Oferta, Perfil, ItemGuardado, AlertaPrecio y DecisionIA, entre otras. Producto representa artículos de hardware o videojuegos, con atributos como nombre, categoría e imagen; Tienda modela las distintas tiendas con sus datos y, en versiones recientes, incluye un campo logo para mostrar el logotipo en el frontend; Oferta relaciona productos y tiendas, registrando precios, URLs y un indicador disponible que permite aplicar un borrado lógico (soft delete) de ofertas obsoletas; Perfil e ItemGuardado gestionan configuraciones de usuario y productos guardados; AlertaPrecio introduce una relación ternaria entre usuario, producto y tienda; y DecisionIA actúa como caché de decisiones de IA para evitar recomputar emparejamientos repetidos.

En la memoria se acompaña esta descripción con un **diagrama SQL** que representa las tablas y sus relaciones. En dicho diagrama se muestran, al menos, las siguientes tablas y claves:

Producto (PK `id_producto`), Tienda (PK `id_tienda`), Oferta (PK `id_oferta`, FK a Producto y Tienda), la tabla de usuarios (`auth_user` de Django), Perfil (FK a usuario), ItemGuardado (FK a Perfil y Producto), AlertaPrecio (PK `id_alerta` con tres FKs a usuario, producto y tienda, reflejando la relación ternaria requerida) y DecisionIA (con campos clave para la combinación de consultas y candidatos). Las relaciones 1:N y N:M se indican mediante cardinalidades clásicas en el diagrama entidad–relación.

4.4 API REST y especificación OpenAPI

La fachada REST se implementa con Django REST Framework, apoyándose en los serializers y viewsets definidos en `backend/api/serializers.py` y `backend/api/views.py`, así como en las rutas configuradas en `backend/api/urls.py`. Cada modelo relevante dispone de su correspondiente serializer, que se encarga de transformar las instancias de Django en representaciones JSON y viceversa, mientras que los viewsets exponen operaciones CRUD mediante métodos HTTP estándar (GET, POST, PUT, PATCH y DELETE). El archivo `api/urls.py` registra los distintos routers y endpoints, proporcionando una estructura coherente y semántica para las URLs, que incluye parámetros de ruta, de query y de cuerpo según lo requerido en el proyecto.

Para documentar la API y cumplir con el requisito de incluir una **especificación OpenAPI de la fachada REST**, se ha integrado una herramienta de generación automática de esquemas (como `drf-spectacular` o `drf-yasg`) en el proyecto Django. A través de esta integración se expone un endpoint `/schema/` que devuelve la descripción de la API en formato OpenAPI (JSON o YAML), así como una interfaz interactiva de documentación (Swagger UI o ReDoc) accesible desde una ruta como `/docs/` o similar. En la memoria se incluye una captura de esta documentación, donde se pueden observar los recursos principales (productos, tiendas, ofertas, alertas, configuración de carrito) junto con los métodos soportados, los parámetros de entrada y las estructuras de respuesta, lo que facilita tanto la comprensión de la arquitectura como la posible reutilización del API por terceros.

4.5 Integración con el motor de scraping

Aunque el scraper se ejecuta como un componente separado en la carpeta `backend/scraper_app`, su arquitectura está estrechamente integrada con el backend Django. El archivo `main_crawler.py` actúa como orquestador, utilizando el módulo `shops/hardware` para instanciar diferentes scrapers específicos por tienda (PcComponentes, Coolmod, LifeInformática, Alternate, NeoByte) a través de una factoría basada en la clase abstracta `BaseScraper`. Estos scrapers usan Playwright y los utilitarios definidos en `scraper_app/utils` (`db_manager.py`, `ia_matcher.py`, `stealth.py`, `logger.py`, etc.) para extraer productos y ofertas, aplicar normalizaciones, tomar decisiones de IA y persistir los datos directamente en los modelos de Django.

Gracias a esta integración, cada ejecución del scraper actualiza el catálogo que la API REST expone al frontend, manteniendo la coherencia entre las distintas capas. La arquitectura se ha

diseñado de forma que el motor de scraping pueda lanzarse mediante scripts independientes (por ejemplo, `scripts/run-crawler.ps1`), pero sin duplicar lógica de acceso a datos: toda la información se centraliza en los modelos del backend y se expone al usuario final a través de la API y de las distintas pantallas del frontend.

5. Descripción de datos

En este apartado se describen algunas de las piezas de código más representativas de UltraPresetGaming, haciendo énfasis en el tratamiento de los datos, la lógica de negocio y las estructuras empleadas tanto en el backend como en el frontend. El objetivo es mostrar cómo se manipulan los datos desde que se obtienen mediante scraping hasta que se sirven a través de la API REST y se consumen en la interfaz web.

5.1 Motor de similitud TF-IDF para emparejar productos

Una de las piezas más relevantes del proyecto se encuentra en el módulo `db_manager.py`, donde se implementa un motor de similitud basado en TF-IDF y similitud del coseno. El objetivo de este componente es representar los nombres y descripciones de los productos como vectores numéricos y medir su similitud, de forma que se puedan detectar productos equivalentes entre tiendas distintas.

Para ello se utiliza la clase `TfidfVectorizer` de la biblioteca `scikit-learn`, que transforma una colección de textos en una matriz TF-IDF, ponderando cada término según su frecuencia en el documento y su rareza en el conjunto de documentos. El resultado es una matriz dispersa donde cada fila representa un producto y cada columna, un término del vocabulario. Sobre esta representación se calcula la similitud del coseno entre vectores, empleando utilidades como `cosine_similarity` o `linear_kernel` de `scikit-learn`, que permiten medir el ángulo entre los vectores y, por tanto, su grado de similitud.

A nivel de tipos de datos, el motor TF-IDF trabaja con listas de cadenas de texto (nombres y descripciones normalizadas de productos) como entrada, produce matrices numéricas de tipo flotante y genera puntuaciones de similitud también en formato float, normalmente en el rango

$[0,1]$

$[0,1]$. Estas puntuaciones se comparan con umbrales configurables (por ejemplo, 0,8 o 0,95) para decidir si dos productos deben considerarse equivalentes o no, y los resultados se utilizan tanto para alimentar el comparador como para decidir cuándo es necesario recurrir a la capa de IA más costosa.

5.2 Limpieza y normalización de nombres de producto

Antes de aplicar TF-IDF o IA, es necesario limpiar y normalizar los nombres de los productos, eliminando ruido y extrayendo información relevante como el modelo concreto de una tarjeta gráfica. Esta lógica se concentra igualmente en `db_manager.py`, donde se definen diversas expresiones regulares precompiladas para identificar patrones frecuentes (por ejemplo, modelos de GPU NVIDIA, sufijos de fabricante, tamaños de memoria, etc.).

En términos de tipos, la función de limpieza suele recibir como entrada una cadena (`str`) con el nombre bruto del producto extraído del HTML y devuelve otra cadena normalizada: en minúsculas, sin caracteres superfluos, con espacios unificados y, en muchos casos, con un campo separado donde se almacena el “modelo” detectado. Internamente se utilizan objetos `Pattern` de la librería estándar `re`, compilados una sola vez al inicio del módulo para mejorar el rendimiento, especialmente en escenarios donde se procesan cientos o miles de productos.

Esta fase de normalización es clave para que tanto el TF-IDF como la IA trabajen sobre datos más limpios y homogéneos, reduciendo falsos positivos y mejorando la calidad de los emparejamientos. Desde el punto de vista del flujo de datos, puede representarse mediante un diagrama en el que una entrada de texto “ruidoso” pasa por varias transformaciones (eliminación de símbolos, sustituciones, extracción de modelo) hasta generar una representación canónica lista para vectorizar.

5.3 Caché de decisiones de IA y robustez ante JSON

Otra pieza destacada es el módulo `ia_matcher.py`, que implementa la lógica de interacción con modelos de lenguaje locales a través de Ollama para validar si dos productos representan el mismo artículo. Este módulo combina varias estrategias: procesamiento por lotes (`batching`) de candidatos, uso de un modelo ligero como filtro previo, un modelo más pesado solo para casos dudosos y almacenamiento en caché de las decisiones en el modelo `DecisionIA`.

El modelo `DecisionIA` se define en `models.py` con campos de tipo `CharField` y `TextField` para almacenar, por ejemplo, la descripción del producto de referencia, la del candidato y el resultado de la decisión (por ejemplo, “igual”, “distinto” o “duda”), así como posibles metadatos adicionales. A nivel de tipos, los atributos de entrada de las funciones del `matcher` suelen ser cadenas (`str`) y listas de cadenas, mientras que la salida es típicamente una lista de valores booleanos o etiquetas categóricas que indican si cada candidato es aceptado o descartado.

Dado que los modelos de lenguaje pueden devolver respuestas JSON no perfectamente formateadas, en `ia_matcher.py` se incluyen funciones auxiliares para validar la estructura, corregir pequeños errores de formato y normalizar valores antes de su uso. Esto implica trabajar con tipos `dict` y `list` en Python, detectando claves obligatorias y corrigiendo inconsistencias. En caso de error irreparable, se recurre bien a un modelo alternativo, bien a la intervención asistida (por ejemplo, a través de `interactive_prompt.py`). Este flujo puede

representarse mediante un diagrama de decisión que muestre las rutas “respuesta válida”, “respuesta corregible” y “respuesta inválida con fallback”.

5.4 Guardado masivo en base de datos con `bulk_create` y `bulk_update`

El proceso de scraping genera grandes volúmenes de datos que deben ser almacenados de forma eficiente en la base de datos. Para ello, en `db_manager.py` y en el propio `main_crawler.py` se utilizan las operaciones `bulk_create` y `bulk_update` del ORM de Django, que permiten insertar o actualizar múltiples filas en una sola consulta al servidor de base de datos.

El patrón habitual consiste en construir listas de instancias de modelos (por ejemplo, objetos `Producto` u `Oferta`) en memoria, a partir de los datos normalizados por el scraper. Estas listas son de tipo `list[Model]` y se pasan a `Model.objects.bulk_create(lista, batch_size=N)` para realizar la inserción en bloques, donde `batch_size` se ajusta para equilibrar el número de filas por consulta con la memoria disponible. Para actualizaciones, se construyen listas de instancias ya existentes con campos modificados y se aplica `bulk_update(lista, ['campo1', 'campo2'], batch_size=N)`.

Este enfoque reduce el número de accesos a base de datos respecto al uso repetitivo de `save()` o `update_or_create()` dentro de bucles, lo que se traduce en una mejora sustancial de rendimiento cuando se procesan centenares o miles de registros. La documentación de Django y diversos recursos sobre optimización confirman que el uso de `bulk_create` y `bulk_update` es una práctica recomendada para operaciones masivas, especialmente en escenarios de ingestión periódica de datos, como un scraper.

5.5 Gestión de carrito y comparador en el frontend

En la capa de frontend destacan los scripts `hardware.js`, `videogames.js` y, especialmente, `prices.js`, donde se implementa la lógica de carrito, selección de ofertas y cálculo de precios finales en la vista de comparación. Estos scripts trabajan con estructuras de datos típicas de JavaScript, como objetos y arrays, y hacen uso tanto de la API REST como del `localStorage` del navegador.

En `prices.js` se define una estructura interna que representa la selección del usuario para cada producto: por ejemplo, un objeto cuyo identificador de producto (`productId`, de tipo numérico o cadena) actúa como clave, y cuyo valor es otro objeto que almacena la oferta elegida (`id` de la oferta, `precio`, `coste de envío`, `tienda`, etc.). Estas estructuras se agrupan en arrays para generar dinámicamente las líneas del checkout. A partir de ahí se calculan totales como el precio oficial acumulado, el ahorro total y el coste final con envío, utilizando operaciones aritméticas básicas sobre números en coma flotante (`number` en JavaScript).

Para los usuarios invitados, el estado del carrito se persiste en `localStorage`, donde los datos se guardan como cadenas JSON mediante `JSON.stringify()` y se recuperan con `JSON.parse()`. Esto

permite que, aunque el usuario recargue la página o cierre y abra el navegador, sus selecciones sigan disponibles hasta que decida vaciar el carrito o finalizar la compra. Desde el punto de vista de flujo de ejecución, este comportamiento puede representarse mediante un diagrama que comience en la lectura de `localStorage` al cargar la página, continúe con la actualización de la interfaz en función de los datos recuperados y termine con la escritura de nuevos valores tras cada interacción del usuario (añadir producto, seleccionar oferta, eliminar elemento, etc.).

5.6 Ejemplo de flujo de datos completo

Resumiendo las piezas anteriores, el flujo completo de datos en `UltraPresetGaming` puede sintetizarse de la siguiente forma:

1. El scraper (`main_crawler.py` y scrapers específicos en `shops/hardware`) extrae información de productos y ofertas desde las distintas tiendas, normaliza nombres y atributos, y construye objetos de modelo en memoria.
2. El módulo `db_manager.py` utiliza TF-IDF y similitud del coseno para agrupar productos similares, apoyándose en expresiones regulares precompiladas y, cuando es necesario, en la capa de IA (`ia_matcher.py`) para resolver casos dudosos, almacenando las decisiones en `DecisionIA`.
3. Las ofertas y productos nuevos o actualizados se guardan en la base de datos utilizando operaciones masivas (`bulk_create` y `bulk_update`), reduciendo el número de consultas y mejorando el rendimiento.
4. El backend expone los datos a través de la API REST de Django, donde serializers y viewsets transforman los modelos en JSON, aplicando filtros como el soft delete de ofertas obsoletas.
5. El frontend consume estos endpoints desde scripts JavaScript, construye listas y comparadores de precios, y gestionan el estado del carrito en memoria y en `localStorage`, ofreciendo al usuario una vista clara de precios, ahorros y opciones disponibles.

Este conjunto de componentes ilustra cómo el proyecto integra distintas técnicas de tratamiento de datos (scraping, normalización, vectorización, IA, operaciones masivas en BD y persistencia en cliente) para ofrecer una solución coherente y eficiente a nivel de arquitectura y de experiencia de usuario.